15. Context: Cancellation and Deadlines

15. Context: Cancellation and Deadlines

Phase: Phase 2 — Core language depth
Estimated time: 1–2 days
Slug: `15-context`

Learning objectives

- Pass context.Context as the first parameter
- Derive cancel and timeout contexts
- Propagate cancellation through goroutines and RPCs
- Never store contexts in structs long-term (usually)

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

context.Context carries deadlines and cancellation across API boundaries. It is how Go services stop work when clients disconnect.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Pass Context as the first parameter: `ctx context.Context`.
- WithCancel / WithTimeout / WithDeadline derive child contexts.
- Listen on ctx.Done(); return ctx.Err() when cancelled.
- Never store Context inside a struct long-term unless carefully justified.
- Values in context are for request-scoped data, not optional parameters.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-context: && cd /tmp/gw-context:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    select {
    case <-time.After(200 * time.Millisecond):
        fmt.Println("work finished")
    case <-ctx.Done():
        fmt.Println(ctx.Err())
    }
}
```

Run it:

```
go run .
```

Expected output:

```
context deadline exceeded
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Create a WithTimeout and select on Done.
2. Propagate ctx into a helper that sleeps and aborts early.
3. Call cancel as soon as the parent no longer needs work.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Why context?

Distributed and concurrent programs need a signal: “stop working, the caller left”.

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()

select {
case <-ctx.Done():
    return ctx.Err()
case res := <-results:
    return res, nil
}
```

Rules of thumb

1. `ctx` is the first argument to functions that do I/O or wait.
2. Call `cancel` to free resources (use defer).
3. Do not pass `nil` context — use `context.Background()` or `TODO()` at edges.
4. Values in context are for request-scoped data (trace IDs), not optional parameters.

Cancellation is cooperative: your code must check `ctx.Done()` or call APIs that do.

Failure modes to watch while coding

- Starting goroutines without tying them to ctx
- Using context for mandatory function parameters
- Forgetting defer cancel()

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- No direct Python stdlib equivalent as pervasive as Context.
- Timeouts on requests should flow down — same idea as ctx.
- Avoid context.Value for required params — use arguments.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

When a handler returns, cancel outstanding downstream calls so goroutines do not leak.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Context: Cancellation and Deadlines” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Starting goroutines without tying them to ctx
- Using context for mandatory function parameters
- Forgetting defer cancel()

Exercises

Work in `exercises/15-context/`. Read that folder's `README.md` before you code.

What you will build

Honor context cancellation/deadlines in the exercise function.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/15-context/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does WithTimeout return?
2. Why is cancel required?
3. What error do you get when timed out?

Further reading

- https://go.dev/blog/context
- https://pkg.go.dev/context